

LATENT PLANNING VIA EMBEDDING ARITHMETIC: A CONTRASTIVE APPROACH TO STRATEGIC REASONING

Anonymous authors
Anonymous submission

ABSTRACT

Planning in high-dimensional decision spaces is increasingly framed through the lens of learned representations, yet the prevailing paradigm still couples those representations to trained policies, value heads, and explicit dynamics models. We dispense with all three and demonstrate that planning can be performed directly within an evaluation-aligned embedding space. We introduce SOLIS, which acquires such a space through supervised contrastive learning: outcome similarity is encoded as geometric proximity, and a single global *advantage vector* orients the representation along a coherent losing-to-winning axis. Action selection thereby collapses to elementary vector arithmetic, as candidate continuations are ranked by their alignment with this direction. Instantiated in chess—a deliberately demanding testbed—SOLIS attains strong play from a shallow, embedding-guided search: it exceeds 2500 Elo under blitz time controls and, at matched search depth, equals or surpasses a configured Stockfish baseline in the majority of equal-depth matchups (nine of twelve) while expanding a strictly bounded and far smaller number of nodes per move. These findings establish evaluation-aligned latent planning as a lightweight, compute-efficient alternative to rollout-based dynamics models and policy learning—an approach that, because it depends only on a learnable state evaluation and the geometry it induces, extends naturally to other perfect-information domains in which state evaluations are available or learnable.

1 INTRODUCTION

Planning in high-dimensional state spaces remains a central challenge in artificial intelligence. Conventional reinforcement learning methods rely on explicit search or learned policies, and both scale poorly as the action space grows: the number of candidate continuations, and with it the cost of deliberation, expands rapidly with problem size (Jiang et al., 2023; Gieslmann and Pokorný, 2022). A prominent line of work confronts this difficulty through latent planning, which reasons over compact learned representations rather than raw states, thereby curbing this combinatorial complexity and supporting more human-like decision-making (Hafner et al., 2019; 2020; Schrittwieser et al., 2020). What has remained unexamined is whether the representation alone—absent any learned transition model or predictive head—can carry the planning computation itself.

We answer this question in the affirmative: a learned embedding space can directly support planning once decision-making is recast as interpolation between regions of advantage. To this end, we introduce **SOLIS** (*Strategic Optimization via Latent Interpolative Spaces*), a chess engine built on a transformer encoder trained with supervised contrastive learning to embed states into an evaluation-aligned latent space. Within this space, embedding proximity tracks similarity of outcome and directional displacement corresponds to progress toward favorable states. SOLIS exploits this geometry by ranking candidate moves according to their alignment with a single global “advantage vector” that orients the latent space along a losing-to-winning axis, thereby reducing the act of planning to a projection in representation space.

Even at modest search depths, SOLIS delivers strong results. A search of only 5 ply under 50 ms/move blitz conditions suffices to reach an Elo above 2500; in equal-depth matches at search depth $S \in \{3, 4, 5\}$ with a fixed branching factor, SOLIS equals or surpasses a configured Stockfish baseline in the majority of matchups while expanding a strictly bounded and substantially smaller number of nodes per move. Latent visualizations of real games corroborate this account: game

trajectories trace smooth, interpretable paths through the embedding space, rendering the planning process not merely effective but legible.

These contributions mark a decisive departure from prior art. Whereas established latent planning methods couple a learned dynamics model with policy or value heads (Hafner et al., 2019; 2020; Schrittwieser et al., 2020), SOLIS eliminates all three and grounds action selection entirely in the geometry of an evaluation-aligned representation. This reduction is the central novelty of our work: it establishes that competitive strategic reasoning can emerge from the structure of the embedding space itself, rather than from an auxiliary predictive head or an explicit forward model. We adopt chess as a demanding testbed precisely because its exhaustively studied search machinery sets a stringent bar; the vector-based formulation, however, is domain-agnostic in principle and extends to other perfect-information settings in which state evaluations are available or learnable.

2 BACKGROUND

Planning in very large action spaces is a formidable challenge, because the number of candidate action sequences grows exponentially with each additional decision step—a combinatorial expansion that any practical method must confront explicitly. This section establishes the conceptual foundations on which SOLIS is built. We first formalize sequential decision-making as a Markov decision process; we then situate our approach with respect to planning, reinforcement learning, and existing chess engines; and we finally review the contrastive-learning objective that shapes the embedding space at the core of our method.

2.1 MARKOV DECISION PROCESSES

Sequential decision-making tasks are formalized as Markov decision processes (MDPs), each specified by the tuple $\{\mathcal{S}, \mathcal{A}, T, R, p(s_0), \gamma\}$, in which $\mathcal{S}$ denotes the state space, $\mathcal{A}$ the action space, T the transition model, R the reward function, $p(s_0)$ the initial-state distribution, and $\gamma \in [0, 1]$ the discount factor applied to future rewards (Puterman, 2014). A policy $\pi : \mathcal{S} \rightarrow p(\mathcal{A})$ is a mapping from states to probability distributions over actions, and the objective of decision-making is to select actions that maximize the expected discounted return.

Within this framework, a *model* denotes reversible access to the MDP dynamics: given a state–action pair, it predicts the resulting next state and reward (Moerland et al., 2022). For games such as chess and Go, the rules furnish this model exactly and at no learning cost, whereas in domains with unknown dynamics the model must be approximated from data (Gieslmann and Pokorný, 2022). This distinction is decisive for our setting: chess supplies a perfect transition model while leaving the far harder problem of *evaluating* states unresolved—and it is precisely this residual evaluation problem that SOLIS targets.

Planning and reinforcement learning are distinguished precisely by how they exploit access to the MDP dynamics (Moerland et al., 2022). Planning methods presuppose a model and simulate forward from the current state to select an action, in a manner analogous to human mental deliberation (Russell and Norvig, 1995); because the resulting solution is computed relative to the current state, it is *local* and is discarded once the action is executed. Reinforcement learning methods, by contrast, typically lack reversible access to the dynamics (Sutton and Barto, 1998): they interact with the environment step by step and, in consequence, learn a *global solution*—a policy or value function—that generalizes across the entire state space. SOLIS occupies a deliberate and principled middle ground, pairing the exact model afforded by the rules of chess with a globally learned representation that supplies the evaluations planning requires.

Model-based reinforcement learning unifies these two elements, employing a model while simultaneously learning a global solution. Following the taxonomy of Moerland et al. (2022), it assumes two forms: model-based RL with a *learned* model (e.g., Dyna (Sutton, 1991)), in which both the dynamics and the global solution are estimated from data, and model-based RL with a *known* model (e.g., AlphaZero (Silver et al., 2017)), in which the dynamics are given and only the global solution is learned. SOLIS belongs to this second, known-model category, but departs sharply from its predecessors in the form the learned component takes: not a policy or value estimator, but the geometry of an evaluation-aligned representation.

2.1.1 MODEL-BASED RL WITH A LEARNED MODEL

A substantial body of work falls under model-based RL with a learned model, in which both the dynamics and the global solution are estimated jointly. PlaNet (Hafner et al., 2019) learns a latent dynamics model and couples it with a learned global solution. Dreamer (Hafner et al., 2020) and its successors DreamerV2 (Hafner et al., 2022) and DreamerV3 (Hafner et al., 2024) extend this paradigm to progressively more complex tasks while retaining global value functions and policies. MuZero (Schrittwieser et al., 2020) likewise learns a latent dynamics model, but integrates it with tree search to strengthen planning alongside a jointly trained policy and value function. Dyna (Sutton, 1991) and MBPO (Janner et al., 2021) further exemplify the pattern, each employing a learned model to support value or policy estimation. Across this literature a learned predictive head or dynamics model is treated as indispensable—the very assumption SOLIS discards.

2.1.2 MODEL-BASED RL WITH A KNOWN MODEL

When the environment dynamics are available, the algorithm can concentrate on coupling search with learning. AlphaZero (Silver et al., 2017) adopts the exact game rules as its model and combines Monte Carlo tree search (MCTS) with learned policy and value heads. SAVE (Hamrick et al., 2020) likewise assumes a simulator and interleaves Q-learning with MCTS: a learned Q-function serves as a search prior, while the search-derived Q-estimates are amortized back into the network. In both cases the learned component takes the form of an explicit policy or value estimator that is repeatedly consulted during search—precisely the component that SOLIS supplants with a single directional operation in a contrastively learned space.

2.1.3 LATENT PLANNING

Latent planning simplifies decision-making by operating in a compressed representation space rather than the raw state space. PlaNet (Hafner et al., 2019), Dreamer (Hafner et al., 2020), and MuZero (Schrittwieser et al., 2020) pair latent dynamics models with value or policy heads, enabling planning through simulated rollouts in the learned space. Expansive Latent Space Trees (ELAST) (Gieslmann and Pokorný, 2022) recast latent planning as exploratory tree search, combining contrastive representations with sampling-based expansion to tackle long-horizon control. The Trajectory Autoencoding Planner (TAP) (Jiang et al., 2023) pursues a related strategy, learning a compact latent action space and planning over discrete action codes. What unites all of these methods is a common dependence: each relies on rollouts, learned transitions, or auxiliary predictive heads to convert the representation into a plan.

SOLIS breaks with this shared design. It trains neither a policy nor a value head and learns no dynamics model; instead, it distills the evaluation targets directly into the geometry of the representation. Planning then proceeds by traversing between regions aligned with favorable outcomes, drawing solely on the embedding space at inference time. This is a strictly more economical use of learning than that of prior latent planners, every one of which requires an additional predictive component to render the latent space actionable—economy that our experiments show comes at no cost in playing strength.

2.2 CHESS ENGINES

IBM’s Deep Blue marked a watershed for computer chess in 1997, defeating the reigning world champion, Garry Kasparov, in a head-to-head match. Early versions of Stockfish (Romstad et al., 2008) inherited its essential design: handcrafted evaluation functions paired with deep alpha-beta tree search. This methodology proved powerful enough to attain superhuman performance, yet its ceiling was ultimately fixed by a human factor—the combined chess mastery of the experts who engineered the static evaluation function.

This rating bottleneck, rooted in the limits of hand-designed evaluation, motivated a line of research aimed at removing human guidance from the evaluation function itself. AlphaZero (Silver et al., 2017) and its open-source counterpart Leela Chess Zero (Pascutto and Linscott, 2018) achieved a decisive advance: engines that learn to evaluate positions entirely through self-play reinforcement learning, with no prior knowledge beyond the rules. Modern versions of Stockfish pursued a complementary route, adopting an efficiently updatable neural-network evaluation module (NNUE) (Nasu,

2018) to accelerate position evaluation while retaining traditional alpha-beta search. In each case learning sharpened the *evaluation*, yet the surrounding search machinery was left substantially intact.

Although these advances have produced engines that decisively surpass human players, their underlying search algorithms have gone largely unchanged. They still rest on principles first articulated by v. Neumann (1928) and Shannon (1950) and later embodied in Deep Blue, relying on deep alpha-beta search over millions of states (Lai, 2015). SOLIS directly challenges this entrenched reliance on broad, deep search: we show that contrastive learning enables a far more economical search process in which planning is carried out as traversal through a high-dimensional embedding space rather than as exhaustive enumeration of the game tree.

2.3 CONTRASTIVE LEARNING

Contrastive representation learning trains a model to place similar inputs close together and dissimilar inputs far apart in a latent space (Chen et al., 2020). Formally, given a batch I of training examples containing an anchor input x_i , an encoder $f(\cdot)$ maps the anchor to a normalized embedding $z_i = f(x_i)$. A positive pair (z_i, z_j) links semantically similar inputs, whereas every remaining pair drawn from the batch is treated as a negative. The learned geometry is therefore governed entirely by which pairs are designated positive—the very degree of freedom that SOLIS repurposes to encode chess evaluations into the representation.

A common contrastive objective is InfoNCE (van den Oord et al., 2019), defined as:

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k \in A(i)} \exp(\text{sim}(z_i, z_k)/\tau)}, \quad (1)$$

where $A(i) = I \setminus \{i\}$ is the set of all batch samples excluding the anchor i , $\text{sim}(\cdot, \cdot)$ denotes a similarity metric, and τ is a scalar temperature that controls the sharpness of the resulting distribution. In the self-supervised regime the positive z_j is an augmentation of the anchor z_i , so that no labels are required (Chen et al., 2020; Khosla et al., 2021).

When labels are available, the supervised contrastive loss (SupCon) (Khosla et al., 2021) generalizes InfoNCE by admitting multiple positive examples per anchor, and is defined as:

$$\mathcal{L}_{\text{sup}} = \sum_{i \in I} \frac{-1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(\text{sim}(z_i, z_p)/\tau)}{\sum_{a \in A(i)} \exp(\text{sim}(z_i, z_a)/\tau)}, \quad (2)$$

where $P(i)$ denotes the set of positives associated with anchor i and $A(i)$ is defined as above. Enlarging the pool of positive and negative pairs has been shown to improve representation quality (Khosla et al., 2021; Chowdhury et al., 2024)—a property that SOLIS exploits directly by deriving positives from evaluation similarity rather than from data augmentation, thereby transferring the scalar ordering of Stockfish evaluations into the geometry of the embedding.

3 METHODS

We now specify the components of our approach in the order in which they compose into a working engine: the tokenization of chess positions, the transformer encoder architecture, the supervised contrastive training procedure that shapes the evaluation-aligned space, and the embedding-guided action selection performed at inference. Each stage is deliberately minimal, and together they realize the central design commitment of this work—that planning can be discharged by the geometry of a learned representation alone, without a policy, value head, or learned dynamics model.

3.1 INPUT REPRESENTATION AND TOKENIZATION

Each chess position is represented by its Forsyth–Edwards Notation (FEN) ASCII string, which compactly encodes piece placement, the side to move, castling rights, the en passant target, the half-move clock, and the full-move counter. Following the tokenization scheme of Ruoss et al. (2024), we convert each FEN into a fixed-length sequence of 77 tokens by expanding its run-length encodings, yielding the uniform input format required by a transformer encoder. As Ruoss et al. (2024)

observe, this per-position tokenization renders chess non-Markovian: information about prior states is discarded, even though such history can be decisive in special cases such as threefold repetition or the fifty-move rule. This is a deliberate design choice whose consequences we characterize precisely, and whose remedy we identify, in Section 5.

3.2 ENCODER ARCHITECTURE

Our encoder is a multi-layer transformer (Vaswani et al., 2023) adapted to tokenized chess positions. We instantiate two variants, SOLIS Base and SOLIS Mini, which span a roughly fifty-fold range in parameter count and thereby let us characterize precisely how representation capacity trades off against search depth and width. Their configurations are summarized in Table 1.

Table 1: Transformer configurations for the two SOLIS variants evaluated in this work. SOLIS Mini (0.8M parameters) and SOLIS Base (41M parameters) share layer depth but differ in embedding dimension, attention-head count, and MLP width, together spanning a roughly fifty-fold range in representation capacity that anchors our capacity-versus-search analysis.

Model	Layers	Embedding dim. D	Heads	MLP Size	Params
SOLIS Mini	6	128	8	256	0.8M
SOLIS Base	6	1024	16	1024	41M

Each input sequence is mapped to D -dimensional vectors through a learned token-embedding matrix. A special classification token (CLS) (Devlin et al., 2019; Dosovitskiy et al., 2021) is prepended to the sequence and serves to aggregate information across the entire position. Because the inputs are of fixed length, we add a learned positional encoding to every token embedding (Ruoss et al., 2024). The augmented sequence is then processed by stacked transformer encoder layers employing GELU (Hendrycks and Gimpel, 2023) activations with dropout regularization (Srivastava et al., 2014). Finally, the hidden state at the CLS position is extracted, passed through a linear projection, and ℓ_2 -normalized to yield the unit-norm embedding z on which every subsequent similarity and advantage computation operates. Confining all downstream geometry to this single normalized vector is what permits action selection to reduce to an inner product against a fixed direction.

3.3 SUPERVISED CONTRASTIVE TRAINING

We train the encoder on 5 million positions sampled uniformly at random from the ChessBench dataset (Ruoss et al., 2024), each annotated with a Stockfish-evaluated win probability for the player to move. All evaluations are normalized to White’s perspective, so that values near 1.0 denote a decisive advantage for White and values near 0.0 a decisive advantage for Black. This scalar target induces a total ordering over positions, and it is exactly this ordering that the contrastive objective transfers into the embedding geometry.

Positives are defined through an evaluation margin δ that determines when two positions count as similar. For each anchor we precompute the set of positions whose win probabilities lie within δ of it and, during training, sample five of them as positives. The batch-level positive mask is constructed analogously: every pair of samples whose evaluation difference falls below δ is marked positive, and all remaining pairs serve as negatives. This margin-based construction pulls together states of nearly equal evaluation while pushing apart positions separated by larger evaluation gaps. Because similarity in the target space is monotone in win probability, the resulting embedding necessarily acquires a coherent global ordering from losing to winning positions—precisely the structure that the advantage vector later exploits, and the mechanism by which planning becomes a linear operation in representation space.

We optimize the supervised contrastive (SupCon) objective (Khosla et al., 2021), using these batch masks to determine positive and negative pairs. Pairwise scores are computed as cosine similarities, with temperature $\tau = 0.07$ (Girdhar et al., 2023; Radford et al., 2021) governing the sharpness of the induced distribution. Optimization uses stochastic gradient descent with momentum 0.9 (Dosovitskiy et al., 2021; Chen et al., 2020; He et al., 2015) and batch size 128 for 400,000 steps, distributed across six NVIDIA L40S GPUs (48 GB each) via data parallelism (Paszke et al., 2019). The complete set of training hyperparameters is reported in Appendix A.2.

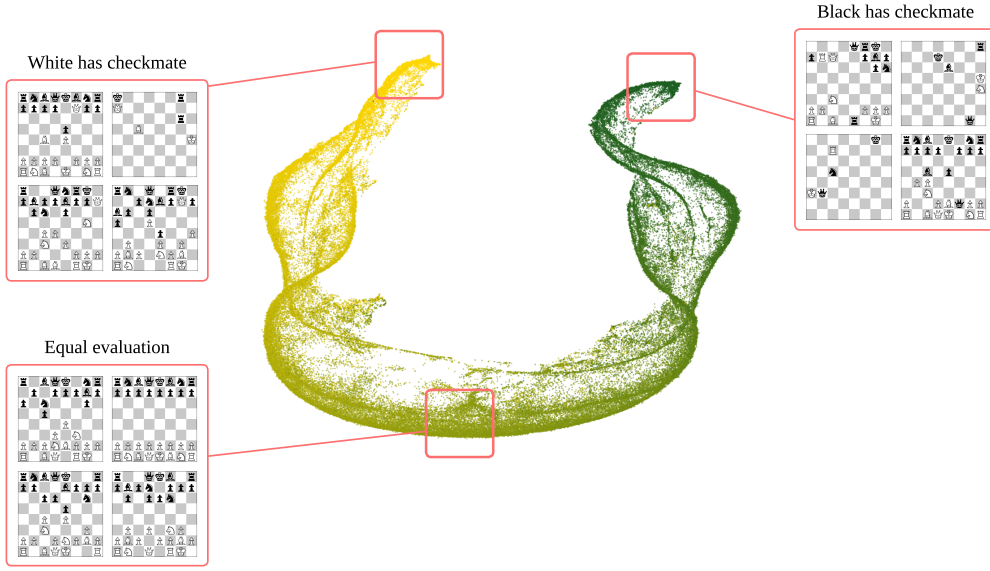


Figure 1: UMAP projection of the SOLIS Base embedding space, with each encoded test position colored by its Stockfish win probability (gold = White favored, green = Black favored). The smooth, monotone color gradient confirms that supervised contrastive training organizes the space into a coherent losing-to-winning continuum—precisely the linear structure that the advantage vector traverses at inference.

Figure 1 visualizes the learned representation of the Base model through a 2D UMAP projection (McInnes et al., 2020). Each point is an encoded test position, colored by its Stockfish win probability for White: positions in which White holds a decisive advantage appear in gold (*White has checkmate*), and those in which Black holds a decisive advantage in dark green (*Black has checkmate*). The projection exhibits a smooth, monotone arrangement of positions along the win-probability axis, providing direct evidence that the contrastive objective has recovered the intended global ordering rather than an arbitrary clustering—the empirical precondition for the vector-arithmetic planning that follows.

3.4 EMBEDDING-GUIDED SEARCH

At inference, SOLIS plans directly in the learned space, ranking legal continuations by their alignment with a single global advantage direction. This direction is estimated exactly once, offline, from the extremes of evaluation on a held-out dataset, and it remains fixed thereafter—so that per-move deliberation incurs no additional learned computation. We first compute the class means

$$\mu_{\text{White}} = \mathbb{E}[z \mid p=1.0], \quad \mu_{\text{Black}} = \mathbb{E}[z \mid p=0.0],$$

the embedding centroids of decisively won and decisively lost positions, and define the normalized advantage vector

$$\vec{a} = \frac{\mu_{\text{White}} - \mu_{\text{Black}}}{\|\mu_{\text{White}} - \mu_{\text{Black}}\|},$$

a fixed unit vector that orients the space from losing to winning regions. From this single direction we derive two complementary mechanisms for mapping a candidate embedding to a scalar score.

3.4.1 UNANCHORED (GLOBAL-DIRECTIONAL)

The first mechanism scores each child embedding by its raw alignment with the advantage direction. A candidate embedding z' receives the score

$$\text{score}_{\text{dir}}(z') = \langle z', \vec{a} \rangle,$$

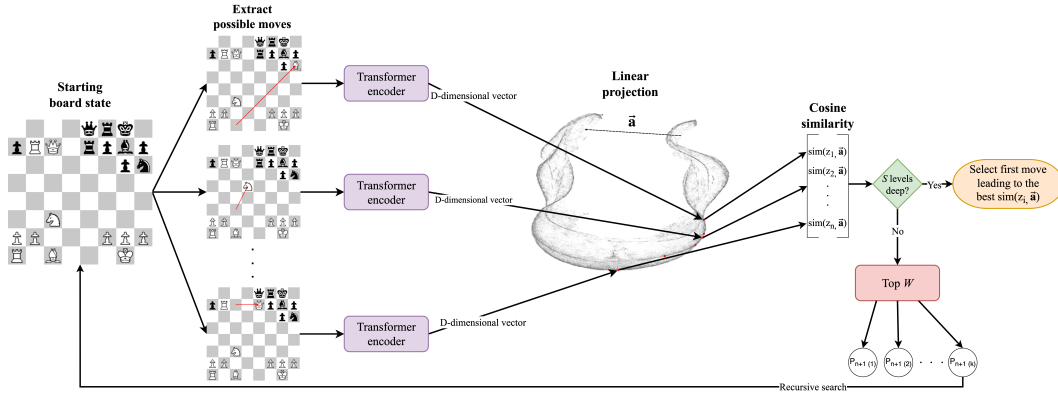


Figure 2: System overview of SOLIS. Legal continuations are embedded in parallel by the transformer encoder and scored by their alignment with the global advantage vector; the top- W children are then expanded recursively to depth S under min-max preference. For clarity the diagram illustrates the unanchored scoring mechanism $\cos(z', \vec{a})$; the anchored variant simply substitutes $\text{score}_{\text{anch}}$, leaving the search procedure unchanged.

which favors continuations whose embeddings project furthest along $\vec{a}$, independent of any reference origin. This is the scoring mechanism depicted in Figure 2.

3.4.2 ANCHORED (RELATIVE PROJECTION)

The second mechanism first translates each embedding by the Black mean and then projects, yielding

$$\text{score}_{\text{anch}}(z') = \langle z' - \mu_{\text{Black}}, \vec{a} \rangle,$$

so that the score equals the signed distance from μ_{Black} along $\vec{a}$. By referencing a principled origin—the centroid of decisively lost positions—rather than the arbitrary coordinate origin, anchoring eliminates a spurious offset and, as our experiments confirm, furnishes a consistently more stable and better-calibrated basis for comparing continuations.

The surrounding search procedure (Figure 2) is identical for both scoring mechanisms, so that any difference in playing strength is attributable to the scoring rule alone. From the “Starting board state” we “Extract possible moves” and embed all children in parallel with the “Transformer encoder.” Each child is then scored by either $\text{score}_{\text{dir}}$ or $\text{score}_{\text{anch}}$ in the “Cosine similarity” block, and only the “Top- W ” children are retained, bounding the branching factor to W . Upon reaching “ S levels deep?” the recursion terminates and we “Select first move...”; otherwise the “Recursive search” continues under min-max preference, with White maximizing and Black minimizing the advantage score. To avoid redundant evaluation of positions reached by transposition, we maintain a transposition table keyed by Zobrist hashes (Zobrist, 1990).

4 RESULTS

4.1 RATING CALCULATION

We estimate Elo ratings by evaluating each model against Stockfish under a fixed per-move time budget of 50 ms (Ruoss et al., 2024); the complete Stockfish configurations appear in Appendix A.3. To disentangle the separate contributions of search breadth and depth, we conducted a controlled ablation across both axes. At each depth/width configuration, every SOLIS model played more than 600 games with alternating colors against Stockfish instances set to a range of Elo caps, and both planning mechanisms—unanchored and anchored—were evaluated under this identical protocol. The caps were deliberately chosen so that each configuration yielded at least one matchup with a positive and one with a negative win rate, bracketing the model’s true strength and thereby ensuring reliable estimation. Ratings and their 95% confidence intervals were then obtained by Bayesian logistic regression using the BayesElo (Coulom, 2008) program with the default confidence parameter

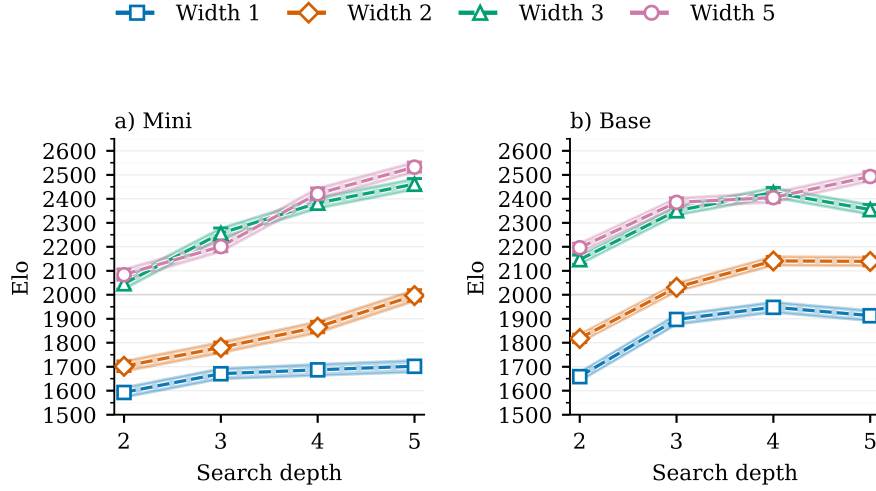


Figure 3: Estimated Elo of SOLIS Mini and Base against time-capped Stockfish across search widths W and depths S , under the *unanchored* planning mechanism. Ratings scale monotonically with search, with depth contributing by far the largest gains and width saturating beyond $W=3$. Shaded bands denote 95% confidence intervals.

of 0.5. Figure 3 reports the unanchored estimates and Figure 4 the anchored estimates; the full complement of Stockfish rating caps and per-matchup results is tabulated in Appendix A.6 for complete reproducibility.

Unanchored ratings Under unanchored planning, Elo scales monotonically with search, and depth contributes by far the largest gains. Width exhibits a pronounced jump from $W \in \{1, 2\}$ to $W=3$ but saturates thereafter, with $W=5$ adding little. This plateau is diagnostic rather than incidental: it demonstrates that the strongest continuation almost always lies within the top three candidates, so that broader expansion yields sharply diminishing returns and $W=3$ is sufficient. At shallow depths SOLIS Base leads Mini by roughly 100–250 Elo, exactly as its larger capacity predicts; at depths 4 and 5, however, Mini closes the gap and even overtakes Base at $W \in \{3, 5\}$ under this mechanism—direct evidence that, once search is deep enough, the compact model’s representation already suffices for strong play, and that SOLIS’s strength resides in representation quality rather than in raw parameter count.

Anchored ratings Anchored scoring produces markedly smoother scaling in both depth and width while preserving a consistent 100–250 Elo margin of Base over Mini. Both models surpass 2500 Elo at depth 5; crucially, and in sharp contrast to the unanchored case, Base dominates Mini across *every* setting, restoring the capacity ordering that theory predicts. Anchoring additionally delivers a steady 10–30 Elo improvement for Base, directly corroborating the analysis of Section 3.4: referencing the centroid of decisively lost positions rather than the arbitrary coordinate origin supplies a demonstrably more reliable basis for evaluation.

4.2 EQUAL-DEPTH SIMULATIONS

We next evaluate SOLIS in head-to-head matches against Stockfish at matched search depths—a setting that removes the time cap and pits the two search procedures directly against one another, isolating the quality of the search itself. Guided by the scaling analysis of Section 4.1, we fix SOLIS’s branching factor at $W=3$, the width beyond which wider search yielded only marginal gains precisely because the strongest continuations reliably fall within the top three candidates. Stockfish, by contrast, retains its default expansion with no imposed branching limit, so any parity that SOLIS achieves is achieved under a strictly narrower search—a deliberately conservative comparison that can only understate SOLIS’s efficiency. At each depth $S \in \{3, 4, 5\}$ we simulate 200 games with alternating colors; results for both scoring mechanisms are reported in Table 2.

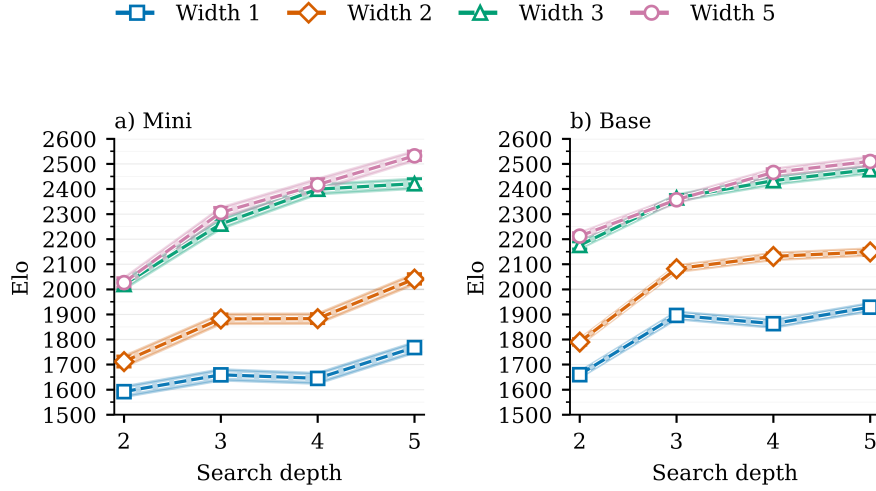


Figure 4: Estimated Elo of SOLIS Mini and Base against time-capped Stockfish across search widths W and depths S , under the *anchored* planning mechanism. Ratings rise smoothly and monotonically with both depth and width; both models exceed 2500 Elo at depth 5, and Base remains uniformly stronger than Mini across every setting. Shaded bands denote 95% confidence intervals.

Simulation results The equal-depth matches establish that SOLIS is competitive with, and frequently superior to, Stockfish at matched depth. Under unanchored scoring, SOLIS Base outscores Stockfish at every tested depth (106, 109, and 107.5 points out of 200 at depths 3, 4, and 5, respectively), while Mini is comparable at depths 3 and 5 and trails only at depth 4. Under anchored scoring, Base concedes a narrow margin at depth 3 (98 to 102) but overtakes Stockfish at depths 4 and 5 (108 and 110 points), and Mini prevails at depths 3 and 5, again dipping below only at depth 4. The single consistent shortfall—Mini at depth 4—is therefore the lone exception to an otherwise uniformly favorable pattern that holds across both scoring mechanisms and both model sizes, with SOLIS taking the higher score in nine of the twelve matchups.

Table 2: Match points from 200-game equal-depth matches of SOLIS against Stockfish at matched depths $S \in \{3, 4, 5\}$, with SOLIS fixed to branching factor $W=3$ and Stockfish using unbounded expansion. The middle pair of columns reports the direct-similarity (unanchored) mechanism and the rightmost pair the anchored-projection mechanism; within each pair, the SOLIS and Stockfish scores sum to 200. Higher is better and draws count as 0.5; the stronger score in each row is shown in bold. SOLIS attains the higher score in nine of the twelve matchups—despite searching a strictly narrower tree than its unbounded opponent.

Config	Depth	Direct Similarity		Anchored Projection	
		SOLIS score	Stockfish score	SOLIS score	Stockfish score
Base	3	106	94	98	102
	4	109	91	108	92
	5	107.5	92.5	110	90
Mini	3	108	92	105	95
	4	85	115	91	109
	5	101	99	113	87

Nodes searched The efficiency gap underlying this parity is substantial and, crucially, provable. With a fixed branching factor $W=3$, the maximum number of nodes SOLIS expands at depth S is bounded in closed form by $N(S) = \sum_{i=0}^S 3^i$, so that $N(3) = 40$, $N(4) = 121$, and $N(5) = 364$. Stockfish, whose expansion varies with pruning and heuristics, searched on average 137 nodes at depth 3 (maximum 1625), 214 at depth 4 (maximum 4463), and 338 at depth 5 (maximum 10611),

reaching well into the thousands in the worst case. SOLIS therefore matches this Stockfish configuration while expanding a small and strictly bounded fraction of the nodes—fewer than the Stockfish average at depths 3 and 4, comparable at depth 5, and one to two orders of magnitude fewer than its worst-case expansion. This substantiates the paper’s central empirical claim: **much of the strength conventionally obtained through broad tree expansion can instead be recovered directly from a sufficiently high-quality learned representation.**

4.3 INTERPRETABLE GAME TRAJECTORIES

Beyond its strength, the learned space confers a degree of interpretability that rollout- and value-head-based planners do not readily afford. Figure 5 traces latent move sequences from three representative games—one won by White, a balanced draw, and one won by Black—each plotted over the same 2D projection. Positions are embedded independently and connected by arrows that mark the temporal progression of play. The resulting trajectories exhibit a clear and consistent structure: decisive games trace smooth paths that migrate steadily toward the corresponding winning region, whereas the drawn game oscillates near the neutral center. Read together with the static embedding of Figure 1, these trajectories render the planning process legible, exhibiting directly how movement through the space corresponds to the accumulation of advantage or disadvantage over the course of a game—an account of *why* a move is chosen that predictive-head planners cannot supply.

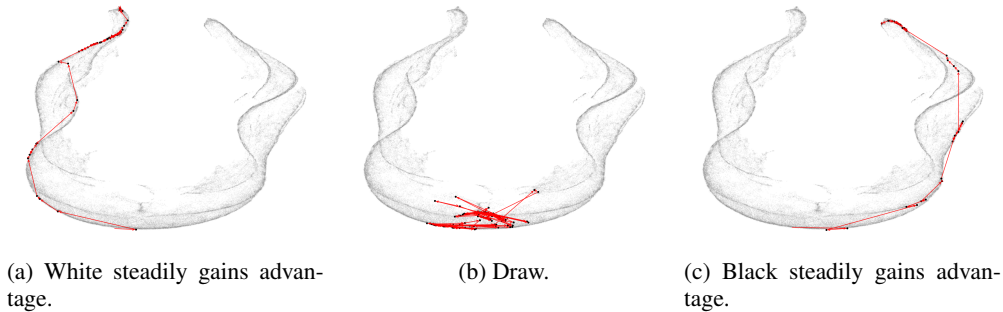


Figure 5: Latent trajectories of three representative games embedded in the shared SOLIS representation space. Red arrows indicate the progression of positions as each game unfolds. Decisive games (a, c) trace smooth paths toward the respective winning region, while the drawn game (b) remains near the neutral center—direct evidence that the geometry of the embedding faithfully tracks the evolving evaluation of the game.

5 DISCUSSION

5.1 LIMITATIONS

Blindness to threefold repetition SOLIS selects moves purely by alignment with the advantage vector, and it is exactly this reduction that makes decision-making effective at minimal overhead; the same economy, however, entails discarding move history, and this omission carries a concrete cost. The most consequential is blindness to threefold repetition: because a position recurring three times—together with the side to move and the castling and en-passant rights—permits either player to claim a draw, an opponent aware of this can engineer repetitions, for instance through perpetual checks, to neutralize a position that SOLIS would otherwise convert. Game C in Appendix A.4 illustrates precisely this failure mode, in which SOLIS reaches a winning position against Stockfish yet concedes a premature draw by repetition. Importantly, this limitation is a direct and fully anticipated consequence of the non-Markovian tokenization introduced in Section 3, not a defect of the planning mechanism itself, and it is readily addressable by augmenting the state with rule-aware history features.

Indecisiveness in winning positions A second limitation arises when many continuations are equally winning. Because both a mate-in-1 and a mate-in-20 carry a win probability of 1.0, the evaluation-aligned space cannot discriminate among forcing lines by length, even though the shorter

is plainly preferable. Selecting a longer continuation leaves the opponent room to complicate the position, which can occasionally squander a win into a draw or even a loss. Notably, Ruoss et al. (2024) encountered the same difficulty and circumvented it by consulting Stockfish as an external oracle; SOLIS instead commits to progressing entirely under its own representation, and we regard closing this gap—by biasing selection toward shorter forced wins—as a well-defined avenue for future work rather than an obstacle to the approach. Game 1 in Appendix A.5 exemplifies the issue, showing SOLIS holding many winning continuations over successive moves yet choosing suboptimally and settling for a draw.

5.2 CONCLUSION

We set out to determine whether planning can be carried out directly in an evaluation-aligned latent space, without training a policy, a value head, or a dynamics model, and our results answer this question decisively in the affirmative. We introduced SOLIS, which learns a contrastive embedding in which proximity encodes outcome similarity and a single global *advantage vector* orients the space along a losing-to-winning axis. Planning thereby reduces to vector arithmetic: candidate actions are ranked by their alignment with this direction and expanded through a small, bounded search.

In chess, SOLIS operates with a fixed branching factor $W=3$ and shallow depths $S \in \{3, 4, 5\}$. Across these settings it equals or surpasses a time-capped Stockfish in the majority of equal-depth matchups—nine of twelve—while expanding a strictly bounded 40–364 nodes per move against Stockfish’s hundreds on average and thousands in the worst case at the same depths. Elo scales with depth, and under anchored scoring both Mini and Base exceed 2500 Elo at depth five, with anchoring contributing a consistent 10–30 Elo gain for Base. Width confers little benefit beyond three candidates, which localizes SOLIS’s strength squarely in the quality of the learned representation rather than in breadth of search. The same embedding renders play interpretable: decisive games advance steadily toward the winning region, while draws linger near neutral.

Taken together, these results establish evaluation-aligned latent planning as a simple, compute-efficient, and interpretable alternative to rollout-based search and explicit dynamics models. Because the method depends only on a learnable state evaluation and the geometry it induces, the same formulation applies naturally to other perfect-information games and to control tasks with large action spaces, where the value of a low-branching, representation-driven search is greatest. Several directions follow directly from the analysis above: incorporating rule-aware state features to resolve the repetition and history limitations, biasing selection toward shorter forced wins, integrating SOLIS with policy priors or MCTS, characterizing the calibration and uncertainty of the advantage direction, and empirically testing transfer beyond chess.

REPRODUCIBILITY STATEMENT

We have documented every component required to reproduce our results in full. The complete training setup, including all hyperparameters and implementation details, is provided in Appendix A.2. Section 3 specifies the encoder architecture, input representation, supervised contrastive objective, and inference procedure, with model configurations summarized in Table 1. The evaluation protocol—match setup, Stockfish configurations, and rating calculation—is detailed in Section 4 and Appendix A.3, and the full per-matchup results underlying every Elo estimate are tabulated in Appendix A.6 for independent verification.

REFERENCES

- T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. A simple framework for contrastive learning of visual representations, 2020. URL <https://arxiv.org/abs/2002.05709>.
- S. Chowdhury, G. Hamerly, and M. McGarrity. Active learning strategy using contrastive learning and k-means for aquatic invasive species recognition. In *2024 IEEE/CVF Winter Conference on Applications of Computer Vision Workshops (WACVW)*, pages 848–858, 2024. doi: 10.1109/WACVW60836.2024.00097.

594 R. Coulom. Whole-history rating: A bayesian rating system for players of time-varying strength.
595 In *Computers and Games*, volume 5131 of *LNCS*, pages 113–124. Springer, 2008. doi: 10.1007/
596 978-3-540-87608-3_10.

597 J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional trans-
598 formers for language understanding, 2019. URL <https://arxiv.org/abs/1810.04805>.

600 A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani,
601 M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words:
602 Transformers for image recognition at scale, 2021. URL [https://arxiv.org/abs/2010.](https://arxiv.org/abs/2010.11929)
603 11929.

604 R. Gieslmann and F. T. Pokorny. Latent planning via expansive tree search. In A. H. Oh, A. Agar-
605 wal, D. Belgrave, and K. Cho, editors, *Advances in Neural Information Processing Systems*, 2022.
606 URL <https://openreview.net/forum?id=zSdz5scsnzU>.

607 R. Girdhar, A. El-Nouby, Z. Liu, M. Singh, K. V. Alwala, A. Joulin, and I. Misra. Imagebind: One
608 embedding space to bind them all, 2023. URL <https://arxiv.org/abs/2305.05665>.

610 D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson. Learning latent
611 dynamics for planning from pixels, 2019. URL <https://arxiv.org/abs/1811.04551>.

612 D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi. Dream to control: Learning behaviors by latent
613 imagination, 2020. URL <https://arxiv.org/abs/1912.01603>.

614 D. Hafner, T. Lillicrap, M. Norouzi, and J. Ba. Mastering atari with discrete world models, 2022.
615 URL <https://arxiv.org/abs/2010.02193>.

616 D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse domains through world models,
617 2024. URL <https://arxiv.org/abs/2301.04104>.

618 J. B. Hamrick, V. Bapst, A. Sanchez-Gonzalez, T. Pfaff, T. Weber, L. Buesing, and P. W.
619 Battaglia. Combining q-learning and search with amortized value estimates, 2020. URL
620 <https://arxiv.org/abs/1912.02807>.

621 K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition, 2015. URL
622 <https://arxiv.org/abs/1512.03385>.

623 D. Hendrycks and K. Gimpel. Gaussian error linear units (gelus), 2023. URL [https://arxiv.](https://arxiv.org/abs/1606.08415)
624 org/abs/1606.08415.

625 M. Janner, J. Fu, M. Zhang, and S. Levine. When to trust your model: Model-based policy opti-
626 mization, 2021. URL <https://arxiv.org/abs/1906.08253>.

627 Z. Jiang, T. Zhang, M. Janner, Y. Li, T. Rocktäschel, E. Grefenstette, and Y. Tian. Efficient planning
628 in a compact latent action space, 2023. URL <https://arxiv.org/abs/2208.10291>.

629 P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan.
630 Supervised contrastive learning, 2021. URL <https://arxiv.org/abs/2004.11362>.

631 M. Lai. Giraffe: Using deep reinforcement learning to play chess, 2015. URL [https://arxiv.](https://arxiv.org/abs/1509.01549)
632 org/abs/1509.01549.

633 L. McInnes, J. Healy, and J. Melville. Umap: Uniform manifold approximation and projection for
634 dimension reduction, 2020. URL <https://arxiv.org/abs/1802.03426>.

635 T. M. Moerland, J. Broekens, A. Plaat, and C. M. Jonker. Model-based reinforcement learning: A
636 survey, 2022. URL <https://arxiv.org/abs/2006.16712>.

637 Y. Nasu. Efficiently updatable neural-network-based evaluation functions for computer shogi, 2018.
638 Accessed: 2024-04-25.

639 G.-C. Pascutto and G. Linscott. Leelachesszero, 2018. URL <https://lczero.org>. Accessed:
640 2024-04-25.

- A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. Pytorch: An imperative style, high-performance deep learning library, 2019. URL <https://arxiv.org/abs/1912.01703>.
- M. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley Series in Probability and Statistics. Wiley, 2014. ISBN 9781118625873. URL <https://books.google.com/books?id=VvBjBAAAQBAJ>.
- A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision, 2021. URL <https://arxiv.org/abs/2103.00020>.
- T. Romstad, M. Costalba, J. Kiiski, G. Linscott, Y. Nasu, M. Isozaki, and H. Noda. Stockfish, 2008. URL <https://stockfishchess.org>. Accessed: 2025-04-22.
- A. Ruoss, G. Delétang, S. Medapati, J. Grau-Moya, L. K. Wenliang, E. Catt, J. Reid, C. A. Lewis, J. Veness, and T. Genewein. Amortized planning with large-scale transformers: A case study on chess, 2024. URL <https://arxiv.org/abs/2402.04494>.
- S. Russell and P. Norvig. *Artificial Intelligence, A modern approach*. 1995.
- J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel, T. Lillicrap, and D. Silver. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, Dec. 2020. ISSN 1476-4687. doi: 10.1038/s41586-020-03051-4. URL <http://dx.doi.org/10.1038/s41586-020-03051-4>.
- C. E. Shannon. XXII. programming a computer for playing chess. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314):256–275, 1950.
- D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm, 2017. URL <https://arxiv.org/abs/1712.01815>.
- N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *J. Mach. Learn. Res.*, 15(1):1929–1958, Jan. 2014. ISSN 1532-4435.
- R. S. Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *ACM Sigart Bulletin*, 2(4):160–163, 1991.
- R. S. Sutton and A. G. Barto. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.
- J. v. Neumann. Zur theorie der gesellschaftsspiele. *Mathematische annalen*, 100(1):295–320, 1928.
- A. van den Oord, Y. Li, and O. Vinyals. Representation learning with contrastive predictive coding, 2019. URL <https://arxiv.org/abs/1807.03748>.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need, 2023. URL <https://arxiv.org/abs/1706.03762>.
- A. L. Zobrist. A new hashing method with application for game playing. *ICGA Journal*, 13(2): 69–73, 1990.

A APPENDIX

A.1 USE OF LARGE LANGUAGE MODELS (LLMs)

Large language models were used solely for minor auxiliary tasks, such as formatting and figure and table placement. They were not employed to design experiments, analyze results, or generate technical content; every methodological and experimental contribution was developed and verified exclusively by the authors.

A.2 TRAINING SETUP

Both SOLIS configurations (Mini and Base) are trained with supervised contrastive learning on 5M ChessBench positions, using cosine similarity over ℓ_2 -normalized embeddings and SGD with momentum on six 48 GB L40S GPUs for 400k steps at batch size 128.

Of these settings, the temperature τ and the positive margin δ are the most consequential, and we selected both empirically. The temperature $\tau = 0.07$ follows established contrastive practice (Radford et al., 2021; Girdhar et al., 2023), under which smaller values sharpen the similarity distribution while larger ones oversmooth the gradients; sweeping $\tau \in \{0.05, 0.07, 0.1\}$, we found 0.07 to converge most reliably across both model sizes. The margin $\delta = 0.05$ balances the competing failure modes of collapsing too many positions into a single neighborhood (large δ) against fragmenting the space (small δ), and it additionally coincides with the noise floor of the Stockfish evaluations, for which sub-0.05 differences are rarely meaningful in practice.

Table 3: Complete SOLIS training hyperparameters, shared across the Mini and Base configurations.

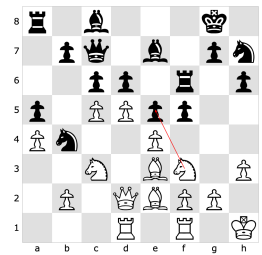
Attribute	Value
Loss	SupCon
τ	0.07
δ (positive margin)	0.05
Positives per anchor	5
Optimizer	SGD
Momentum	0.9
Learning rate	0.05
Batch size	128
Steps	400k
Dropout	0.10
Activation	GELU
Similarity	cosine
Embedding norm	ℓ_2
Token length	77

A.3 STOCKFISH SETUP

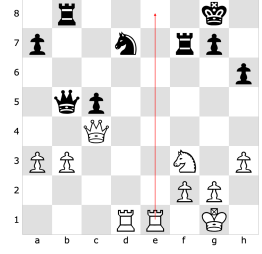
Across all experiments we use the official Linux release of Stockfish 16 (Romstad et al., 2008) with a 32 MB hash and default threading. Rating caps are imposed through the `UCI_LimitStrength` and `UCI_Elo` options, while fixed-depth play is configured through the `depth` parameter. To record the number of nodes searched per move, we set the engine information level to `INFO_ALL` and extract the reported `nodes` attribute.

A.4 REPRESENTATIVE GAMES: STOCKFISH (CAP) VS. SOLIS (DEPTH, WIDTH)

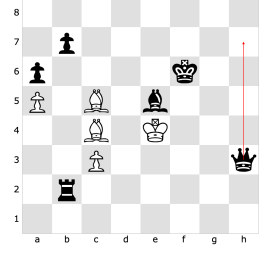
We present representative games between SOLIS and Stockfish, with SOLIS restricted to search depth S and width W while Stockfish searches without a depth bound under a rating cap. Games *A*, *B*, and *E* showcase SOLIS's tactical understanding and long-term planning, whereas Games *C* and *D* isolate the specific failure modes analyzed in Section 5. In each table the left portion records the moves in Portable Game Notation (PGN) and the right portion depicts a critical moment of the game.

<pre> 1. d4 Nf6 2. c4 d6 3. Nc3 e5 4. Nf3 Qe7 5. e4 c6 6. d5 Qc7 7. Be2 h6 8. a4 a5 9. O-O Be7 10. h3 Na6 11. Be3 Nb4 12. Kh1 Nh7 13. Qd2 O-O 14. Rad1 f5 15. c5 Rf6 16. Nxe5 f4 17. Bxf4 Rxf4 18. Qxf4 dxe5 19. d6 exf4 20. Bc4+ Kf8 21. dxc7 Ke8 22. e5 Ng5 23. Rd6 Bxd6 24. exd6 Bd7 25. Re1+ Kf8 26. Re7 Bf5 27. h4 Nd5 28. Nxd5 cxd5 29. Bxd5 Be6 30. Bxe6 f3 31. hxg5 b6 32. Bh3 hxg5 33. Re5 Rc8 34. Rf5+ Kg8 35. Rf7 g4 36. cxb6 g6 37. d7 fxg2+ 38. Kh2 Rf8 39. Bxg2 Kxf7 40. Bc6 Ke7 41. Kg3 Rf3+ 42. Kh2 Rd3 43. b7 Rh3+ 44. Kg2 Re3 45. d8=Q+ Kf7 46. b8=Q Rg3+ 47. Kh2 Rh3+ 48. Kg1 Rh7 49. Qe8+ Kf6 50. Qbd8+ Kf5 51. Be4+ Kf4 52. Qf6# 1-0 </pre>	
--	---

Game A — SOLIS ($S=5$, $W=3$) vs. SF 2350, SOLIS–White. Result: 1–0. SOLIS sacrifices a knight in the middlegame to secure a lasting positional advantage, which it then converts into a win.

<pre> 1. d4 d5 2. e3 c6 3. Bd3 h6 4. h3 Nf6 5. Nf3 e6 6. O-O b6 7. c4 Ba6 8. b3 Nbd7 9. Bb2 Bb4 10. a3 Be7 11. Nbd2 O-O 12. Re1 c5 13. Qe2 Qc7 14. e4 Nxe4 15. Nxe4 dxe4 16. Bxe4 Rab8 17. d5 Bf6 18. dxe6 Bxb2 19. exf7+ Rxf7 20. Qxb2 Bb7 21. Bxb7 Qxb7 22. Rad1 b5 23. Qe2 bxc4 24. Qxc4 Qb5 25. Re8+ Nf8 26. Rxb8 Qxb8 27. Rd5 Qf4 28. Qxf4 Rxf4 29. Rxc5 Ne6 30. Rc6 Nd4 31. Nxd4 a5 32. Rc4 Rf8 33. Nc6 Kh8 34. Nxa5 Ra8 35. b4 Rd8 36. g3 Rd3 37. a4 g5 38. b5 Rd2 39. b6 Rd5 40. b7 Rxa5 41. Rc8+ Kh7 42. b8=Q Rd5 43. Rc7+ Kg6 44. Qg8+ Kf5 45. Qxd5+ Kg6 46. Qf7# 1-0 </pre>	
--	--

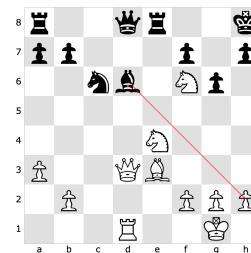
Game B — SOLIS ($S=5$, $W=3$) vs. SF 2300, SOLIS–White. Result: 1–0. SOLIS sacrifices a rook to win Stockfish's queen, then converts the resulting material advantage into a win.

<pre> 1. e3 Nf6 2. Nc3 d5 3. Nf3 c5 4. d4 e6 5. g3 a6 6. a4 Nc6 7. Ne2 Bd6 8. Bg2 O-O 9. h3 Re8 10. c3 Qc7 11. Nd2 e5 12. Nb3 c4 13. dxe5 Nxe5 14. Nbd4 Nd3+ 15. Kf1 Nc5 16. Bf3 Bd7 17. Kg2 Be5 18. a5 Rad8 19. Qc2 g6 20. Bd2 h5 21. Be1 h4 22. gxh4 Kg7 23. Ng3 Rh8 24. h5 Rdg8 25. hxg6 fxg6 26. Be2 Qd6 27. Qd1 Qe7 28. f3 Bd6 29. Nf1 Rh5 30. h4 Rgh8 31. Bf2 R5h7 32. Qe1 Bb8 33. Bg3 Ba7 34. Bf2 Nh5 35. Ng3 Bb8 36. f4 Nf6 37. h5 Rh6 38. hxg6 Rxh1 39. Qxh1 Qe8 40. Ndf5+ Bxf5 41. Nxf5+ Kxg6 42. Ne7+ Kf7 43. e4 Rxh1 44. Rxh1 Qxe7 45. Bxc5 Qxe4+ 46. Bf3 Qg6+ 47. Kf1 Bxf4 48. b4 cxb3 49. Rh8 Ke6 50. Rf8 b2 51. Rxf6+ Kxf6 52. Bxd5 b1=R+ 53. Ke2 Rb2+ 54. Kf3 Be5 55. Bc4 Qh5+ 56. Ke3 Qh3+ 57. Ke4 Qh7+ 58. Kf3 Qh3+ 59. Ke4 Qh7+ 60. Kf3 Qh3+ 61. Ke4 Qh7+ 62. Kf3 Qh3+ 63. Ke4 Qh7+ 64. Kf3 Qh3+ 65. Ke4 1/2-1/2 </pre>	
--	---

Game C — SF 2200 vs. SOLIS ($S=5$, $W=3$), SOLIS–Black. Result: 1/2–1/2. SOLIS reaches a winning endgame yet settles for a draw, exhibiting the threefold-repetition blindness analyzed in Section 5.

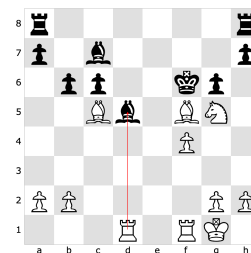
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863

1. d4 d5 2. c4 dxc4 3. e3 e5 4. Bxc4 exd4 5. exd4
Nf6 6. Nc3 Bd6 7. Qe2+ Qe7 8. Be3 O-O 9. Nf3 c6 10.
O-O Be6 11. Rfd1 Re8 12. d5 cxd5 13. Bxd5 Nc6 14.
a3 Nxd5 15. Nxd5 Qd8 16. Qd3 Bg4 17. Ng5 g6 18. Ne4
Bxd1 19. Ndf6+ Kh8 20. Rxd1 Bxh2+ 21. Kxh2 Qxd3 22.
Rxd3 Red8 23. Nd7 Kg7 24. f3 h6 25. Rd6 f5 26. Nec5
Re8 27. Bd2 Re2 28. Kh1 Kf7 29. b3 Ke7 30. Nxb7 Rc8
31. Ndc5 Kf7 32. Kg1 Kg8 33. Rxg6+ Kf7 34. Rd6 Ne7
35. Rd7 Rg8 36. Nd6+ Kg6 37. Nc4 Kh5 38. Ne3 f4 39.
Ne6 fxe3 40. Nf4+ Kg5 41. Nxe2 exd2 42. Kf2 Kf6 43.
Nc3 Ke6 44. Rd4 Rc8 45. Ne4 d1=N+ 46. Rxd1 Rc2+ 47.
Kg1 Nf5 48. Rd2 Rxd2 49. Nxd2 Ke5 50. Kh2 Kf4 51.
b4 Ke3 52. Ne4 Nd4 53. a4 Nc2 54. b5 Nd4 55. Nd6
Ne6 56. a5 Kd3 57. g4 Kd4 58. b6 axb6 59. axb6 Nc5
60. Kg3 Kd5 61. b7 Na6 62. Nf7 Kc6 63. Nxh6 Kxb7
64. f4 Kc7 65. Kh4 Kd7 66. Kh5 Nc5 67. f5 Kd6 68.
f6 Ne6 69. Nf7+ Kc6 70. Kh6 Kd5 71. g5 Nc5 72. Kg6
Ne6 73. Kh5 Kd4 74. g6 Kc4 75. Ne5+ Kd5 76. Ng4 Ke4
77. Kh6 Kf5 78. g7 Nxg7 79. Kxg7 Kxg4 80. Kf7 Kf5
81. Ke7 Kg5 82. f7 Kf4 83. Kf6 Kf3 84. Ke5 Kg3 85.
Ke4 Kf2 86. Kd3 Kg1 87. Ke3 Kh2 88. f8=Q Kg3 89.
Qg8+ Kh3 90. Kf4 Kh2 91. Kf3 Kh3 92. Qg3# 1-0



Game D — SF 2500 vs. SOLIS ($S=5$, $W=3$), SOLIS–Black. Result: 1-0 SOLIS commits a short-sighted piece sacrifice that yields a lasting material disadvantage and, ultimately, a loss.

1. d4 d5 2. e3 Nd7 3. c4 e5 4. dxe5 dxc4 5. Nf3
Qe7 6. Bxc4 Nxe5 7. Nxe5 Qxe5 8. Qb3 Qh5 9. e4
Nf6 10. O-O Bd6 11. Qb5+ c6 12. Qxh5 Nxh5 13. Nc3
Be5 14. f4 Bc7 15. Be3 g6 16. Bb3 Be6 17. Rad1 Ng7
18. Bc2 f5 19. Bc5 Kf7 20. exf5 Nxf5 21. Ne4 b6
22. Ng5+ Kf6 23. Bxf5 Bd5 24. Rxd5 cxd5 25. Bd4+
Kxf5 26. h3 Bxf4 27. Bxh8 Kxg5 28. Bg7 Rg8 29. Bd4
Re8 30. Rf3 Re1+ 31. Kf2 Re6 32. g3 Bd2 33. Rf7
Re4 34. Bf6+ Kh6 35. g4 a6 36. Kg3 d4 37. h4 Be1+
38. Kh3 Re3+ 39. Kg2 Bxh4 40. Bxh4 a5 41. Bf6 a4
42. Rd7 Rd3 43. Rd5 b5 44. Rxb5 a3 45. Re5 Rd2+
46. Kf3 axb2 47. g5+ Kh5 48. Rb5 Rd3+ 49. Kf4 h6
50. gxh6+ Kxh6 51. Rxb2 g5+ 52. Kg4 Rd1 53. Bxg5+
Kg6 54. Rb6+ Kg7 55. Kf5 Kf7 56. Rb7+ Ke8 57. a4
Ra1 58. Re7+ Kf8 59. Rh7 Rxa4 60. Bh6+ Kg8 61. Re7
Rc4 62. Kg5 Rc5+ 63. Kg4 Ra5 64. Rg7+ Kh8 65. Re7
d3 66. Rd7 d2 67. Bxd2 Rc5 68. Bf4 Kg8 69. Be3 Rb5
70. Bh6 Re5 71. Kf4 Rc5 72. Ke4 Rb5 73. Kf4 Rb4+
74. Ke5 Rb1 75. Kf6 Rb2 76. Kg6 Rb6+ 77. Kh5 Rb2
78. Bg5 Rb6 79. Bh6 Re6 80. Kg5 Ra6 81. Rg7+ Kh8
82. Re7 Ra5+ 83. Kg6 Ra6+ 84. Kh5 Kg8 85. Rb7 Rc6
86. Rd7 Rc5+ 87. Bg5 Rc2 88. Kg6 Rc6+ 89. Bf6 Re6
90. Rd8+ Re8 91. Rxe8# 1-0

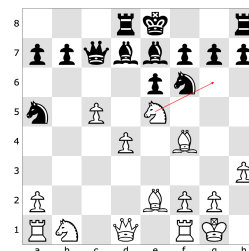


Game E — SOLIS ($S=5$, $W=3$) vs. Stockfish 2500 , SOLIS–White. Result: 1-0 SOLIS makes an exchange sacrifice for long-term compensation, then converts the resulting advantage into a win.

A.5 EQUAL-DEPTH MATCHES: SOLIS *D* vs. STOCKFISH *D*

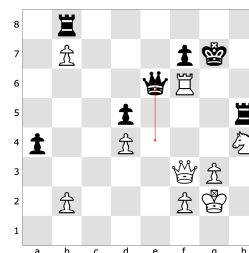
We include several illustrative games in which SOLIS and Stockfish search to the same depth *D*. These examples make concrete how SOLIS's latent planning competes head-to-head with Stockfish's search once no rating cap is imposed. As before, the left portion of each table records the moves in Portable Game Notation (PGN) and the right portion depicts a critical moment of the game.

1. d4 d5 2. e3 c5 3. Nf3 Nf6 4. Be2 e6 5. O-O Nc6 6. b3 cxd4 7. exd4 Be7 8. c3 Qc7 9. h3 Bd7 10. c4 dxc4 11. bxc4 Na5 12. Ne5 Rd8 13. Bf4 Bd6 14. c5 Be7 15. Ng6 Qc8 16. Nxh8 Bc6 17. Nc3 Kf8 18. Rc1 Qa8 19. Bc7 b5 20. Bxa5 Bxg2 21. d5 Bxh3 22. Qd3 Bf5 23. Qg3 b4 24. Rfd1 Bxc5 25. Qc7 Be7 26. Nb5 Nxd5 27. Rxd5 Qxd5 28. Bxb4 Qd7 29. Qxd7 Re8 30. Rc8 g5 31. Qxe7+ Kg7 32. Qxf7+ Kh6 33. Bf8+ Rxf8 34. Qxf8# 1-0



Game F — SOLIS $S = 5$ vs. Stockfish $S = 5$, SOLIS-White. Result: 1-0 SOLIS sacrifices a knight in the middlegame for a concrete tactical advantage and carries that advantage through to a win.

1. d4 d5 2. e3 c5 3. Nf3 e6 4. Be2 cxd4 5. exd4 Bd6 6. O-O Nf6 7. Bg5 Nbd7 8. Nbd2 h6 9. Bh4 O-O 10. Re1 a5 11. Bf1 Bf4 12. c3 Qb6 13. Qc2 Qc7 14. a4 b6 15. Bb5 Ba6 16. Bxa6 Rxa6 17. Qd3 Ra7 18. Re2 Qc8 19. Reel Nh5 20. Nf1 Nh6 21. N1d2 Nh5 22. Nf1 Bc7 23. Qd1 Nh6 24. Qd3 Nh5 25. Qd1 Nh6 26. Qd3 Rb7 27. Qb5 Ra7 28. N1d2 Nh5 29. h3 g5 30. Rac1 gxf4 31. c4 Nf4 32. Rc3 Qd8 33. g3 Nxf3+ 34. Kg2 Ng5 35. cxd5 h3+ 36. Kg1 h2+ 37. Kxh2 exd5 38. Rc6 Nf6 39. Nxf5 hxg5 40. Kg2 Re8 41. Rh1 Bd6 42. Nf3 Kg7 43. Nxf5 Rc7 44. Rxd6 Qxd6 45. Qf1 b5 46. axb5 Rcc8 47. Rh4 Rh8 48. Nf3 Ne4 49. Rf4 Rh5 50. Qe2 Qe6 51. Nh4 Rch8 52. b6 Nf6 53. Qf3 a4 54. b7 Rb8 55. Rxf6 Qe4 56. Rxf7+ Kh8 57. Qxe4 dxe4 58. Rc7 Rg5 59. Rc8+ Rg8 60. Rc7 Rg5 61. Rc8+ Rg8 62. Ng6+ Kg7 63. Ne7 Rf8 64. Rc7 Rf7 65. d5 Rbf8 66. Nf5+ Kg6 67. Rxf7 Kxf7 68. Nd6+ Kg6 69. Nc8 Kg5 70. b8=Q Kg4 71. Qc7 Rf5 72. Nd6 e3 73. Qg7+ Rg5 74. f3+ Kh5 75. Qh7# 1-0



Game G — SOLIS $S = 5$ vs. Stockfish $S = 5$, SOLIS-White. Result: 1-0 Stockfish steadily accumulates an advantage but blunders in the late middlegame; SOLIS capitalizes on the error and prevails in the resulting endgame.

918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971

1. d4 c6 2. e4 d5 3. Nc3 Nf6 4. e5 Ng8 5. Bd3 e6 6. Nf3 c5 7. dxc5 Bxc5 8. O-O Nc6 9. a3 h6 10. b4 Bb6 11. Nb5 Bc7 12. Re1 Bb8 13. c4 Nge7 14. Nd6+ Bxd6 15. exd6 Qxd6 16. Bb2 O-O 17. b5 dxc4 18. Bh7+ Kh8 19. Qxd6 f6 20. Be4 c3 21. Bxc3 e5 22. bxc6 Bf5 23. Qxe7 Bxe4 24. Rxe4 bxc6 25. Nxe5 Rfe8 26. Nf7+ Kg8 27. Nxh6+ Kh7 28. Rael Rad8 29. Ba5 Rd4 30. g4 Rxe4 31. Rxe4 Rb8 32. Qxa7 Rd8 33. Rf4 Rd3 34. g5 fxg5 35. Rd4 Rxa3 36. Kf1 Rb3 37. Rb4 Rc3 38. h4 gxh4 39. Rd4 h3 40. Qe7 h2 41. Qe4+ Kxh6 42. Rd6+ Kg5 43. Rg6+ Kh5 44. Qg4# 1-0	
---	--

Game H — SOLIS $S = 3$ vs. Stockfish $S = 3$, SOLIS–White. Result: 1-0 Even at depth 3, SOLIS sacrifices a bishop for a discovered attack on Stockfish’s queen and converts the resulting advantage into a win.

1. d4 d5 2. e3 c6 3. c4 e6 4. Nf3 Nf6 5. Be2 c5 6. O-O Nc6 7. b3 a6 8. Bb2 Be7 9. a3 cxd4 10. exd4 O-O 11. c5 b6 12. b4 bxc5 13. dxc5 Bb7 14. Nbd2 a5 15. b5 Bxc5 16. Rc1 Qb6 17. bxc6 Ng4 18. Rxc5 Ba6 19. Rc2 Nxf2 20. Rxf2 Bxe2 21. Qxe2 a4 22. c7 Rac8 23. Bd4 Qb5 24. Qxb5 Rfe8 25. Qb8 h6 26. Ne5 g5 27. h4 Rf8 28. Ne4 dxe4 29. Rxf7 Rxb8 30. Rxf8+ Rxf8 31. hxcg5 Rc8 32. gxh6 e3 33. Bxe3 Kh7 34. Rc4 Kg8 35. Kh2 Kh7 36. g4 Rg8 37. g5 Ra8 38. Rg4 Kh8 39. Ba7 Rf8 40. Bb8 Rf2+ 41. Kg1 Rc2 42. Rc4 Rb2 43. Ba7 Rb1+ 44. Kg2 Kh7 45. Bf2 Rb2 46. g6+ Kxh6 47. Kh2 Rxf2+ 48. Kg3 Rg2+ 49. Kxg2 Kg7 50. Rc1 Kf6 51. Kh2 Kg7 52. Nd7 e5 53. Nf6 Kxf6 54. Rg1 Kg7 55. Rf1 Kh6 56. Rg1 Kg7 57. Rf1 Kxg6 58. Rc1 Kg7 59. Rc4 Kf7 60. Kg2 Ke7 61. Kf2 Kd6 62. Ke1 Kd5 63. Rc1 Ke4 64. Ke2 Kd4 65. Rc2 e4 66. Rc1 Kd5 67. Ke1 Ke5 68. Rc4 Kd5 69. Rc2 Kd6 70. Rc1 Ke7 71. Rc4 Kf7 72. Rc5 e3 73. Kf1 Ke7 74. Rc4 Kf7 75. Kg2 e2 76. Kf2 Ke7 77. Ke1 Kf7 78. Rc5 Kg7 79. Rc2 Kf6 80. Rc4 Kf7 81. Rc5 Kg6 82. Kd2 Kf7 83. Ke1 Ke7 84. Rc4 Kf7 85. Rc5 Kg6 86. Rc4 Kf7 87. Rc5 1/2-1/2	
--	--

Game I — SOLIS $S = 3$ vs. Stockfish $S = 3$, SOLIS–White. Result: 1/2-1/2 SOLIS struggles to plan when every continuation is winning, illustrating the indecisiveness in winning positions analyzed in Section 5.

A.6 SOLIS VS. STOCKFISH RESULTS

To render our Elo estimates fully reproducible, we report the complete match outcomes between SOLIS and Stockfish, with SOLIS searching at depths $S \in \{2, 3, 4, 5\}$. Each table is indexed by Stockfish’s Elo cap (rows) and SOLIS’s search width W (columns), and every entry reports the win–loss–draw counts of the matchup together with the corresponding total match points in parentheses. Separate tables are provided for the Base and Mini models and for both the unanchored and anchored scoring mechanisms, so that every reported rating can be traced directly to the games that produced it.

Base: Depth 2					Base: Depth 3				
SF Elo	W=1	W=2	W=3	W=5	SF Elo	W=1	W=2	W=3	W=5
1500	32–46–22 (55)	41–47–12 (64)	–	–	1500	–	–	–	–
1550	39–38–23 (58)	37–50–13 (62)	–	–	1550	–	–	–	–
1600	32–44–24 (54)	33–51–16 (58)	–	–	1600	–	–	–	–
1650	33–41–26 (54)	29–46–25 (52)	–	–	1650	–	–	–	–
1700	34–38–28 (53)	22–53–25 (48)	–	–	1700	–	–	–	–
1750	37–35–28 (54)	30–49–21 (54)	–	–	1750	–	–	–	–
1800	29–42–29 (50)	36–42–22 (57)	–	–	1800	39–29–32 (54)	37–49–14 (62)	–	–
1850	–	–	–	–	1850	35–32–33 (51)	30–50–20 (55)	–	–
1900	–	–	–	–	1900	37–30–33 (52)	40–41–19 (60)	–	–
1950	–	–	–	–	1950	31–28–41 (45)	37–44–19 (59)	–	–
2000	–	–	34–40–26 (54)	43–31–26 (58)	2000	41–27–32 (54)	33–45–22 (56)	–	–
2050	–	–	46–38–16 (65)	36–38–26 (55)	2050	39–27–34 (52)	31–41–28 (52)	–	–
2100	–	–	34–45–21 (56)	34–31–35 (50)	2100	41–27–32 (54)	36–38–26 (55)	–	–
2150	–	–	46–31–23 (62)	30–43–27 (52)	2150	–	–	–	–
2200	–	–	31–35–34 (48)	35–28–37 (49)	2200	–	–	33–42–25 (54)	30–41–29 (50)
2250	–	–	34–34–32 (51)	40–29–31 (54)	2250	–	–	35–41–24 (56)	33–43–24 (54)
2300	–	–	32–32–36 (48)	37–34–29 (54)	2300	–	–	35–33–32 (52)	36–34–30 (53)
2350	–	–	–	–	2350	–	–	37–37–26 (56)	29–37–34 (48)
2400	–	–	–	–	2400	–	–	34–36–30 (52)	37–38–25 (56)
2450	–	–	–	–	2450	–	–	37–28–35 (51)	43–29–28 (58)
2500	–	–	–	–	2500	–	–	41–30–29 (56)	34–30–36 (49)
2550	–	–	–	–	2550	–	–	–	–
2600	–	–	–	–	2600	–	–	–	–
2650	–	–	–	–	2650	–	–	–	–
2700	–	–	–	–	2700	–	–	–	–

Base: Depth 4					Base: Depth 5				
SF Elo	W=1	W=2	W=3	W=5	SF Elo	W=1	W=2	W=3	W=5
1500	–	–	–	–	1500	–	–	–	–
1550	–	–	–	–	1550	–	–	–	–
1600	–	–	–	–	1600	–	–	–	–
1650	–	–	–	–	1650	–	–	–	–
1700	–	–	–	–	1700	–	–	–	–
1750	–	–	–	–	1750	–	–	–	–
1800	–	–	–	–	1800	–	–	–	–
1850	–	–	–	–	1850	–	–	–	–
1900	–	–	–	–	1900	–	–	–	–
1950	37–26–37 (50)	41–42–17 (62)	–	–	1950	36–34–30 (53)	38–41–21 (58)	–	–
2000	43–20–37 (53)	31–43–26 (52)	–	–	2000	31–28–41 (45)	27–52–21 (53)	–	–
2050	38–27–35 (52)	40–38–22 (59)	–	–	2050	41–24–35 (53)	36–37–27 (54)	–	–
2100	42–25–33 (54)	32–48–20 (56)	–	–	2100	43–23–34 (54)	36–39–25 (56)	–	–
2150	36–23–41 (48)	31–47–22 (54)	–	–	2150	42–23–35 (54)	27–45–28 (50)	–	–
2200	46–13–41 (52)	39–39–22 (58)	–	–	2200	42–21–37 (52)	23–51–26 (48)	–	–
2250	–	–	–	–	2250	–	–	–	–
2300	–	–	–	–	2300	–	–	–	–
2350	–	–	–	–	2350	–	–	–	–
2400	–	–	–	–	2400	–	–	–	–
2450	–	–	40–39–21 (60)	39–38–23 (58)	2450	–	–	25–46–29 (48)	43–30–27 (58)
2500	–	–	38–38–24 (57)	41–40–19 (61)	2500	–	–	40–36–24 (58)	34–44–22 (56)
2550	–	–	44–20–36 (54)	42–29–29 (56)	2550	–	–	40–31–29 (56)	38–38–24 (57)
2600	–	–	46–17–37 (54)	37–23–40 (48)	2600	–	–	46–25–29 (58)	33–37–30 (52)
2650	–	–	41–18–41 (50)	44–22–34 (55)	2650	–	–	47–17–36 (56)	30–35–35 (48)
2700	–	–	45–16–39 (53)	42–22–36 (53)	2700	–	–	38–25–37 (50)	45–21–34 (56)

Figure 6: Full match outcomes for SOLIS Base against Stockfish under the unanchored planning mechanism, organized into per-depth blocks and indexed by Stockfish Elo cap (rows) and search width W (columns). Each cell reports win–loss–draw counts with the total match points out of 100 in parentheses. These per-matchup results constitute the complete evidence underlying the Base unanchored Elo estimates.

1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079

Mini: Depth 2					Mini: Depth 3				
SF Elo	W=1	W=2	W=3	W=5	SF Elo	W=1	W=2	W=3	W=5
1350	32–32–36 (48)	27–45–28 (50)	–	–	1350	–	–	–	–
1400	30–41–29 (50)	33–27–40 (46)	–	–	1400	–	–	–	–
1450	29–44–27 (51)	33–40–27 (53)	–	–	1450	–	–	–	–
1500	31–38–31 (50)	31–39–30 (50)	–	–	1500	29–38–33 (48)	28–43–29 (50)	–	–
1550	38–35–27 (56)	23–50–27 (48)	–	–	1550	41–36–23 (59)	36–37–27 (54)	–	–
1600	34–30–36 (49)	29–37–34 (48)	–	–	1600	37–30–33 (52)	37–30–33 (52)	–	–
1650	24–45–31 (46)	28–36–36 (46)	–	–	1650	32–38–30 (51)	30–40–30 (50)	–	–
1700	–	–	–	–	1700	28–37–35 (46)	21–43–36 (42)	–	–
1750	–	–	–	–	1750	36–33–31 (52)	28–40–32 (48)	–	–
1800	–	–	37–31–32 (52)	37–36–27 (55)	1800	39–24–37 (51)	29–35–36 (46)	–	–
1850	–	–	34–24–42 (46)	34–31–35 (50)	1850	–	–	–	–
1900	–	–	40–28–32 (54)	40–29–31 (54)	1900	–	–	–	–
1950	–	–	39–29–32 (54)	39–29–32 (54)	1950	–	–	–	–
2000	–	–	38–34–28 (55)	42–28–30 (56)	2000	–	–	34–35–31 (52)	38–21–41 (48)
2050	–	–	41–27–32 (54)	40–26–34 (53)	2050	–	–	38–34–28 (55)	34–33–33 (50)
2100	–	–	35–30–35 (50)	36–26–38 (49)	2100	–	–	33–34–33 (50)	34–29–37 (48)
2150	–	–	–	–	2150	–	–	31–35–34 (48)	38–25–37 (50)
2200	–	–	–	–	2200	–	–	38–18–44 (47)	31–26–43 (44)
2250	–	–	–	–	2250	–	–	36–23–41 (48)	45–26–29 (58)
2300	–	–	–	–	2300	–	–	40–22–38 (51)	34–26–40 (47)
2350	–	–	–	–	2350	–	–	–	–
2400	–	–	–	–	2400	–	–	–	–
2500	–	–	–	–	2500	–	–	–	–
Mini: Depth 4					Mini: Depth 5				
SF Elo	W=1	W=2	W=3	W=5	SF Elo	W=1	W=2	W=3	W=5
1350	–	–	–	–	1350	–	–	–	–
1400	–	–	–	–	1400	–	–	–	–
1450	–	–	–	–	1450	–	–	–	–
1500	–	–	–	–	1500	–	–	–	–
1550	–	–	–	–	1550	–	–	–	–
1600	36–42–22 (57)	35–31–34 (50)	–	–	1600	–	–	–	–
1650	29–40–31 (49)	36–33–31 (52)	–	–	1650	–	–	–	–
1700	33–34–33 (50)	36–32–32 (52)	–	–	1700	39–30–31 (54)	34–34–32 (51)	–	–
1750	43–26–31 (56)	28–38–34 (47)	–	–	1750	37–29–34 (52)	32–30–38 (47)	–	–
1800	38–24–38 (50)	30–37–33 (48)	–	–	1800	42–20–38 (52)	33–31–36 (48)	–	–
1850	49–23–28 (60)	29–33–38 (46)	–	–	1850	36–24–40 (48)	43–28–29 (57)	–	–
1900	–	–	–	–	1900	45–26–29 (58)	46–31–23 (62)	–	–
1950	–	–	–	–	1950	40–22–38 (51)	37–35–28 (54)	–	–
2000	–	–	–	–	2000	–	–	–	–
2050	–	–	–	–	2050	–	–	–	–
2100	–	–	–	–	2100	–	–	–	–
2150	–	–	31–31–38 (46)	44–21–35 (54)	2150	–	–	–	–
2200	–	–	35–29–36 (50)	48–27–25 (62)	2200	–	–	27–37–36 (46)	33–38–29 (52)
2250	–	–	37–31–32 (52)	37–28–35 (51)	2250	–	–	30–33–37 (46)	38–31–31 (54)
2300	–	–	27–37–36 (46)	36–34–30 (53)	2300	–	–	38–37–25 (56)	33–39–28 (52)
2350	–	–	22–42–36 (43)	37–35–28 (54)	2350	–	–	32–29–39 (46)	29–36–35 (47)
2400	–	–	36–29–35 (50)	37–26–37 (50)	2400	–	–	30–26–44 (43)	30–32–38 (46)
2500	–	–	–	–	2500	–	–	26–34–40 (43)	43–25–32 (56)

Figure 7: Full match outcomes for SOLIS Mini against Stockfish under the unanchored planning mechanism, organized into per-depth blocks and indexed by Stockfish Elo cap (rows) and search width W (columns). Each cell reports win–draw–loss counts with the total match points out of 100 in parentheses. These per-matchup results constitute the complete evidence underlying the Mini unanchored Elo estimates.

1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133

Base: Depth 2					Base: Depth 3				
SF Elo	W=1	W=2	W=3	W=5	SF Elo	W=1	W=2	W=3	W=5
1500	32–46–22 (55)	41–47–12 (64)	–	–	1500	–	–	–	–
1550	39–38–23 (58)	37–50–13 (62)	–	–	1550	–	–	–	–
1600	32–44–24 (54)	33–51–16 (58)	–	–	1600	–	–	–	–
1650	33–41–26 (54)	29–46–25 (52)	–	–	1650	–	–	–	–
1700	34–38–28 (53)	22–53–25 (48)	–	–	1700	–	–	–	–
1750	37–35–28 (54)	30–49–21 (54)	–	–	1750	–	–	–	–
1800	29–42–29 (50)	36–42–22 (57)	–	–	1800	39–29–32 (54)	37–49–14 (62)	–	–
1850	–	–	–	–	1850	35–32–33 (51)	30–50–20 (55)	–	–
1900	–	–	–	–	1900	37–30–33 (52)	40–41–19 (60)	–	–
1950	–	–	–	–	1950	31–28–41 (45)	37–44–19 (59)	–	–
2000	–	–	34–40–26 (54)	43–31–26 (58)	2000	41–27–32 (54)	33–45–22 (56)	–	–
2050	–	–	46–38–16 (65)	36–38–26 (55)	2050	39–27–34 (52)	31–41–28 (52)	–	–
2100	–	–	34–45–21 (56)	34–31–35 (50)	2100	41–27–32 (54)	36–38–26 (55)	–	–
2150	–	–	46–31–23 (62)	30–43–27 (52)	2150	–	–	–	–
2200	–	–	31–35–34 (48)	35–28–37 (49)	2200	–	–	33–42–25 (54)	30–41–29 (50)
2250	–	–	34–34–32 (51)	40–29–31 (54)	2250	–	–	35–41–24 (56)	33–43–24 (54)
2300	–	–	32–32–36 (48)	37–34–29 (54)	2300	–	–	35–33–32 (52)	36–34–30 (53)
2350	–	–	–	–	2350	–	–	37–37–26 (56)	29–37–34 (48)
2400	–	–	–	–	2400	–	–	34–36–30 (52)	37–38–25 (56)
2450	–	–	–	–	2450	–	–	37–28–35 (51)	43–29–28 (58)
2500	–	–	–	–	2500	–	–	41–30–29 (56)	34–30–36 (49)
2550	–	–	–	–	2550	–	–	–	–
2600	–	–	–	–	2600	–	–	–	–
2650	–	–	–	–	2650	–	–	–	–
2700	–	–	–	–	2700	–	–	–	–

Base: Depth 4					Base: Depth 5				
SF Elo	W=1	W=2	W=3	W=5	SF Elo	W=1	W=2	W=3	W=5
1500	–	–	–	–	1500	–	–	–	–
1550	–	–	–	–	1550	–	–	–	–
1600	–	–	–	–	1600	–	–	–	–
1650	–	–	–	–	1650	–	–	–	–
1700	–	–	–	–	1700	–	–	–	–
1750	–	–	–	–	1750	–	–	–	–
1800	–	–	–	–	1800	–	–	–	–
1850	–	–	–	–	1850	–	–	–	–
1900	–	–	–	–	1900	–	–	–	–
1950	37–26–37 (50)	41–42–17 (62)	–	–	1950	36–34–30 (53)	38–41–21 (58)	–	–
2000	43–20–37 (53)	31–43–26 (52)	–	–	2000	31–28–41 (45)	27–52–21 (53)	–	–
2050	38–27–35 (52)	40–38–22 (59)	–	–	2050	41–24–35 (53)	36–37–27 (54)	–	–
2100	42–25–33 (54)	32–48–20 (56)	–	–	2100	43–23–34 (54)	36–39–25 (56)	–	–
2150	36–23–41 (48)	31–47–22 (54)	–	–	2150	42–23–35 (54)	27–45–28 (50)	–	–
2200	46–13–41 (52)	39–39–22 (58)	–	–	2200	42–21–37 (52)	23–51–26 (48)	–	–
2250	–	–	–	–	2250	–	–	–	–
2300	–	–	–	–	2300	–	–	–	–
2350	–	–	–	–	2350	–	–	–	–
2400	–	–	–	–	2400	–	–	–	–
2450	–	–	40–39–21 (60)	39–38–23 (58)	2450	–	–	25–46–29 (48)	43–30–27 (58)
2500	–	–	38–38–24 (57)	41–40–19 (61)	2500	–	–	40–36–24 (58)	34–44–22 (56)
2550	–	–	44–20–36 (54)	42–29–29 (56)	2550	–	–	40–31–29 (56)	38–38–24 (57)
2600	–	–	46–17–37 (54)	37–23–40 (48)	2600	–	–	46–25–29 (58)	33–37–30 (52)
2650	–	–	41–18–41 (50)	44–22–34 (55)	2650	–	–	47–17–36 (56)	30–35–35 (48)
2700	–	–	45–16–39 (53)	42–22–36 (53)	2700	–	–	38–25–37 (50)	45–21–34 (56)

Figure 8: Full match outcomes for SOLIS Base against Stockfish under the anchored planning mechanism, organized into per-depth blocks and indexed by Stockfish Elo cap (rows) and search width W (columns). Each cell reports win–draw–loss counts with the total match points out of 100 in parentheses. These per-matchup results constitute the complete evidence underlying the Base anchored Elo estimates.

1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187

Mini: Depth 2					Mini: Depth 3				
SF Elo	W=1	W=2	W=3	W=5	SF Elo	W=1	W=2	W=3	W=5
1350	36–32–32 (52)	39–30–31 (54)	–	–	1350	–	–	–	–
1400	23–49–28 (48)	30–32–38 (46)	–	–	1400	–	–	–	–
1450	33–37–30 (52)	24–47–29 (48)	–	–	1450	–	–	–	–
1500	34–38–28 (53)	25–40–35 (45)	–	–	1500	29–40–31 (49)	35–33–32 (52)	–	–
1550	28–38–34 (47)	40–39–21 (60)	–	–	1550	34–35–31 (52)	37–29–34 (52)	–	–
1600	23–47–30 (46)	28–42–30 (49)	–	–	1600	32–36–32 (50)	32–39–29 (52)	–	–
1650	35–29–36 (50)	38–37–25 (56)	–	–	1650	34–30–36 (49)	37–33–30 (54)	–	–
1700	–	–	–	–	1700	36–30–34 (51)	34–35–31 (52)	–	–
1750	–	–	–	–	1750	34–25–41 (46)	34–30–36 (49)	–	–
1800	–	–	35–37–28 (54)	40–28–32 (54)	1800	32–29–39 (46)	32–31–37 (48)	–	–
1850	–	–	32–32–36 (48)	44–29–27 (58)	1850	–	–	–	–
1900	–	–	37–30–33 (52)	45–21–34 (56)	1900	–	–	–	–
1950	–	–	43–30–27 (58)	36–29–35 (50)	1950	–	–	–	–
2000	–	–	35–31–34 (50)	43–27–30 (56)	2000	–	–	46–32–22 (62)	38–27–35 (52)
2050	–	–	37–29–34 (52)	45–27–28 (58)	2050	–	–	37–27–36 (50)	39–26–35 (52)
2100	–	–	36–27–37 (50)	42–34–24 (59)	2100	–	–	29–34–37 (46)	48–17–35 (56)
2150	–	–	–	–	2150	–	–	37–33–30 (54)	39–25–36 (52)
2200	–	–	–	–	2200	–	–	37–31–32 (52)	41–23–36 (52)
2250	–	–	–	–	2250	–	–	36–23–41 (48)	44–20–36 (54)
2300	–	–	–	–	2300	–	–	30–43–27 (52)	38–26–36 (51)
2350	–	–	–	–	2350	–	–	–	–
2400	–	–	–	–	2400	–	–	–	–
2500	–	–	–	–	2500	–	–	–	–

Mini: Depth 4					Mini: Depth 5				
SF Elo	W=1	W=2	W=3	W=5	SF Elo	W=1	W=2	W=3	W=5
1350	–	–	–	–	1350	–	–	–	–
1400	–	–	–	–	1400	–	–	–	–
1450	–	–	–	–	1450	–	–	–	–
1500	–	–	–	–	1500	–	–	–	–
1550	–	–	–	–	1550	–	–	–	–
1600	26–45–29 (48)	31–33–36 (48)	–	–	1600	–	–	–	–
1650	37–39–24 (56)	28–32–40 (44)	–	–	1650	–	–	–	–
1700	23–39–38 (42)	29–41–30 (50)	–	–	1700	37–36–27 (55)	34–29–37 (48)	–	–
1750	36–29–35 (50)	38–30–32 (53)	–	–	1750	42–23–35 (54)	37–33–30 (54)	–	–
1800	35–24–41 (47)	33–36–31 (51)	–	–	1800	36–28–36 (50)	31–35–34 (48)	–	–
1850	41–21–38 (52)	30–28–42 (44)	–	–	1850	38–21–41 (48)	33–32–35 (49)	–	–
1900	–	–	–	–	1900	37–27–36 (50)	37–37–26 (56)	–	–
1950	–	–	–	–	1950	43–18–39 (52)	38–24–38 (50)	–	–
2000	–	–	–	–	2000	–	–	–	–
2050	–	–	–	–	2050	–	–	–	–
2100	–	–	–	–	2100	–	–	–	–
2150	–	–	39–23–38 (50)	32–30–38 (47)	2150	–	–	–	–
2200	–	–	33–40–27 (53)	38–25–37 (50)	2200	–	–	30–40–30 (50)	32–34–34 (49)
2250	–	–	36–31–33 (52)	30–27–43 (44)	2250	–	–	29–43–28 (50)	49–26–25 (62)
2300	–	–	32–33–35 (48)	37–28–35 (51)	2300	–	–	31–30–39 (46)	30–35–35 (48)
2350	–	–	35–34–31 (52)	40–25–35 (52)	2350	–	–	26–32–42 (42)	40–34–26 (57)
2400	–	–	28–26–46 (41)	26–38–36 (45)	2400	–	–	39–32–29 (55)	31–32–37 (47)
2500	–	–	–	–	2500	–	–	30–29–41 (44)	28–41–31 (48)

Figure 9: Full match outcomes for SOLIS Mini against Stockfish under the anchored planning mechanism, organized into per-depth blocks and indexed by Stockfish Elo cap (rows) and search width W (columns). Each cell reports win–draw–loss counts with the total match points out of 100 in parentheses. These per-matchup results constitute the complete evidence underlying the Mini anchored Elo estimates.